БОЛЬШИЕ ВЫЗОВЫ 2025-2026

Agent AI

Разработка нейросетевой программной среды с динамической самомодификацией

НАПРАВЛЕНИЕ

Большие данные, ИИ
и информационная безопасность

РАЗРАБОТЧИК

Ролдугин Илья Владимирович
ГБПОУ ВО "ГПК"

Актуальность, Цели и Задачи

Актуальность

Индустрия переходит от простых чат-ботов к **автономным агентным системам**, способным самостоятельно решать сложные задачи.

Проблема

Существующие ИИ-клиенты имеют **статичную архитектуру**: их инструменты жестко заданы разработчиком и не меняются под задачу.

ЦЕЛЕВАЯ АУДИТОРИЯ

IT-специалисты и опытные пользователи

Цель проекта

Создание высокопроизводительного ИИ-агента с архитектурой **динамической самомодификации**.

КЛЮЧЕВЫЕ ЗАДАЧИ

- **Механизм Runtime-исполнения и модификации кода.**
- **Адаптивный UI (пресеты и HTML-вставки).**
- **Взаимодействие ИИ с GUI операционной системы.**
- **Автономное повышение качества кодинга модели.**

Ядро Системы и Базовые Инструменты

MAIN.PY

```python
from agent import Chat

ai = Chat()
ai.set_preset("developer")
ai.set_mode("file_structure", "sandbox")

ai.add_message(f"Задача пользователя: {input()}")
files = ai.ai_get(
    "Выведи список файлов для изменения",
    target_type=list,
    clear_history=False
)
ai.send("Выполни задачу, меняя файлы внутри песочницы")

# ...
```

Центральный узел

Класс **Chat** — это ядро агента. Он управляет API нейросети, хранит историю контекста, предоставляет доступ к инструментам и поддерживает локальное окружение (`self.local_env`).

БАЗОВЫЙ ИНСТРУМЕНТАРИЙ ОС

Инструмент Python

Исполнение сгенерированного кода напрямую в локальном окружении процесса.

Инструмент Shell

Интерактивная сессия терминала с сохранением контекста между вызовами.

Динамическая Самомодификация

АРХИТЕКТУРА ИЗОЛЯЦИИ (SANDBOX)

СЛОВАРЬ LOCAL_ENV

Экземпляр Chat
self

Переменная вывода result = ""

AGENT.PY (УПРОЩЕННАЯ РЕАЛИЗАЦИЯ ФРАГМЕНТА)

```
def python_tool(self, code):
    self.local_env["self"] = self
    self.local_env["result"] = ''
    exec(code, self.local_env, self.local_env)
    return str(self.local_env["result"])
```

ВЫЗОВ ИНСТРУМЕНТА PYTHON

```
import types

# 1. Определение логики инструмента
def custom_math_tool(self, expression):
    # Безопасный вызов eval без builtins
    res = eval(expression, {'__builtins__': {}})
    return f"Результат вычисления: {res}"

# 2. Привязка метода к инстансу Chat
self.custom_math_tool = types.MethodType(
    custom_math_tool, self
)

# 3. Регистрация JSON-схемы для Gemini
self.tools.append({
    # Описание в формате OpenAI API
})
```